SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10 **Total Marks:50**

Annexure A

Debugging or Optimization task

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)

Process P2:
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)

Identify the exact condition that causes deadlock.
Suggest an optimisation to eliminate the deadlock without changing process logic.

(10 Marks)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.
You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready …

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(10 Marks)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB

- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(10 Marks)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(10 Marks)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.
Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(10 Marks)

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1. Deadlock Debugging

CODE:

```
#include <stdio.h>
int main()
{
   int R1 = 0, R2 = 0;
   printf("===== DEADLOCK SIMULATION =====\n\n");
   printf("Process P1 locks Resource R1\n");
   R1 = 1;
   printf("Process P2 locks Resource R2\n");
   R2 = 1;
   printf("Process P1 requests Resource R2...\n");
   if (R2 == 1)
      printf("P1 is Waiting for R2\n");
   printf("Process P2 requests Resource R1...\n");
   if (R1 == 1)
      printf("P2 is Waiting for R1\n");
   printf("\nDeadlock Detected!\n");
   printf("\n===== OPTIMIZED SOLUTION =====\n\n");
   R1 = 0;
   R2 = 0;
   printf("Both processes acquire resources in the same order.\n\n");
   printf("P1 locks R1\n");
```

```
    R1 = 1;
    printf("P1 locks R2\n");
    R2 = 1;
    printf("P1 enters Critical Section\n");
    R2 = 0;
    R1 = 0;
    printf("P1 releases R2\n");
    printf("P1 releases R1\n\n");
    printf("P2 locks R1\n");
    R1 = 1;
    printf("P2 locks R2\n");
    R2 = 1;
    printf("P2 enters Critical Section\n");
    R2 = 0;
    R1 = 0;
    printf("P2 releases R2\n");
    printf("P2 releases R1\n");
    printf("\nDeadlock Eliminated Successfully.\n");
    return 0;
}
```

OUTPUT :

```
===== DEADLOCK SIMULATION =====
Process P1 locks Resource R1
```

Process P2 locks Resource R2

Process P1 requests Resource R2...

P1 is Waiting for R2

Process P2 requests Resource R1...

P2 is Waiting for R1

Deadlock Detected!

===== OPTIMIZED SOLUTION =====

Both processes acquire resources in the same order.

P1 locks R1

P1 locks R2

P1 enters Critical Section

P1 releases R2

P1 releases R1

P2 locks R1

P2 locks R2

P2 enters Critical Section

P2 releases R2

P2 releases R1

Deadlock Eliminated Successfully.

=== Code Execution Successful ===

EXECUTION :

Programiz C Online Compiler — Programiz PRO ›

main.c — Share — Run — Output — Clear

```c
#include <stdio.h>
int main()
{
    int R1 = 0, R2 = 0;
    printf("===== DEADLOCK SIMULATION =====\n\n");
    printf("Process P1 locks Resource R1\n");
    R1 = 1;
    printf("Process P2 locks Resource R2\n");
    R2 = 1;
    printf("Process P1 requests Resource R2...\n");
    if (R2 == 1)
        printf("P1 is Waiting for R2\n");
    printf("Process P2 requests Resource R1...\n");
    if (R1 == 1)
        printf("P2 is Waiting for R1\n");
    printf("\nDeadlock Detected!\n");
    printf("\n===== OPTIMIZED SOLUTION =====\n\n");
    R1 = 0;
    R2 = 0;
```

```
===== DEADLOCK SIMULATION =====

Process P1 locks Resource R1
Process P2 locks Resource R2
Process P1 requests Resource R2...
P1 is Waiting for R2
Process P2 requests Resource R1...
P2 is Waiting for R1

Deadlock Detected!

===== OPTIMIZED SOLUTION =====

Both processes acquire resources in the same order.

P1 locks R1
P1 locks R2
P1 enters Critical Section
P1 releases R2
```

Programiz C Online Compiler — Programiz PRO ›

main.c — Share — Run — Output — Clear

```c
    printf("P1 locks R2\n");
    R2 = 1;
    printf("P1 enters Critical Section\n");
    R2 = 0;
    R1 = 0;
    printf("P1 releases R2\n");
    printf("P1 releases R1\n\n");
    printf("P2 locks R1\n");
    R1 = 1;
    printf("P2 locks R2\n");
    R2 = 1;
    printf("P2 enters Critical Section\n");
    R2 = 0;
    R1 = 0;
    printf("P2 releases R2\n");
    printf("P2 releases R1\n");
    printf("\nDeadlock Eliminated Successfully.\n");
    return 0;
}
```

```
Both processes acquire resources in the same order.

P1 locks R1
P1 locks R2
P1 enters Critical Section
P1 releases R2
P1 releases R1

P2 locks R1
P2 locks R2
P2 enters Critical Section
P2 releases R2
P2 releases R1

Deadlock Eliminated Successfully.


=== Code Execution Successful ===
```

2. CPU Utilisation Problem

CODE :

```
#include <stdio.h>
int main()
{
   int n, i;
   int cpuBurst[20], ioWait[20];
   int totalCPU = 0, totalIO = 0;
   float utilization_before, utilization_after;
   printf("CPU Utilisation Simulation\n");
   printf("-------------------------\n");
   printf("Enter the number of processes: ");
   scanf("%d", &n);
   for(i = 0; i < n; i++)
   {
      printf("\nProcess P%d\n", i + 1);
      printf("Enter CPU Burst Time (ms): ");
      scanf("%d", &cpuBurst[i]);
      printf("Enter I/O Waiting Time (ms): ");
      scanf("%d", &ioWait[i]);
      totalCPU += cpuBurst[i];
      totalIO += ioWait[i];
   }
```

```
    utilization_before = ((float)totalCPU /
                          (totalCPU + totalIO)) * 100;
    printf("\n========== BEFORE OPTIMIZATION ==========\n");
    printf("Total CPU Time      : %d ms\n", totalCPU);
    printf("Total I/O Wait Time : %d ms\n", totalIO);
    printf("CPU Utilisation     : %.2f%%\n", utilization_before);
    printf("\nReason:\n");
    printf("Processes frequently move to Waiting state due to I/O.\n");
    totalIO = totalIO / 2;
    utilization_after = ((float)totalCPU /
                         (totalCPU + totalIO)) * 100;
    printf("\n========== AFTER OPTIMIZATION ==========\n");
    printf("Reduced I/O Wait Time : %d ms\n", totalIO);
    printf("CPU Utilisation       : %.2f%%\n", utilization_after);
    printf("\nOptimization Applied:\n");
    printf("1. Faster I/O Devices\n");
    printf("2. Asynchronous I/O / Increased Multiprogramming\n");
    return 0;
}
```

OUTPUT :

```
CPU Utilisation Simulation
--------------------------
```

Enter the number of processes: 3

Process P1

Enter CPU Burst Time (ms): 10

Enter I/O Waiting Time (ms): 20

Process P2

Enter CPU Burst Time (ms): 34

Enter I/O Waiting Time (ms): 23

Process P3

Enter CPU Burst Time (ms): 20

Enter I/O Waiting Time (ms): 35

========== BEFORE OPTIMIZATION ==========

Total CPU Time : 64 ms

Total I/O Wait Time : 78 ms

CPU Utilisation : 45.07%

Reason:

Processes frequently move to Waiting state due to I/O.

========== AFTER OPTIMIZATION ==========

Reduced I/O Wait Time : 39 ms

CPU Utilisation : 62.14%

Optimization Applied:

1. Faster I/O Devices

2. Asynchronous I/O / Increased Multiprogramming

=== Code Execution Successful ===

EXECUTION :

main.c

```c
#include <stdio.h>
int main()
{
    int n, i;
    int cpuBurst[20], ioWait[20];
    int totalCPU = 0, totalIO = 0;
    float utilization_before, utilization_after;
    printf("CPU Utilisation Simulation\n");
    printf("-------------------------\n");
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        printf("\nProcess P%d\n", i + 1);
        printf("Enter CPU Burst Time (ms): ");
        scanf("%d", &cpuBurst[i]);
        printf("Enter I/O Waiting Time (ms): ");
        scanf("%d", &ioWait[i]);
        totalCPU += cpuBurst[i];
```

Output

```
CPU Utilisation Simulation
-------------------------
Enter the number of processes: 3

Process P1
Enter CPU Burst Time (ms): 10
Enter I/O Waiting Time (ms): 20

Process P2
Enter CPU Burst Time (ms): 34
Enter I/O Waiting Time (ms): 23

Process P3
Enter CPU Burst Time (ms): 20
Enter I/O Waiting Time (ms): 35

========== BEFORE OPTIMIZATION ==========
Total CPU Time      : 64 ms
Total I/O Wait Time : 78 ms
```

main.c

```c
                    (totalCPU + totalIO)) * 100;
    printf("\n========== BEFORE OPTIMIZATION ==========\n");
    printf("Total CPU Time      : %d ms\n", totalCPU);
    printf("Total I/O Wait Time : %d ms\n", totalIO);
    printf("CPU Utilisation     : %.2f%%\n", utilization_before);
    printf("\nReason:\n");
    printf("Processes frequently move to Waiting state due to I/O
        .\n");
    totalIO = totalIO / 2;
    utilization_after = ((float)totalCPU /
                    (totalCPU + totalIO)) * 100;
    printf("\n========== AFTER OPTIMIZATION ==========\n");
    printf("Reduced I/O Wait Time : %d ms\n", totalIO);
    printf("CPU Utilisation       : %.2f%%\n", utilization_after);
    printf("\nOptimization Applied:\n");
    printf("1. Faster I/O Devices\n");
    printf("2. Asynchronous I/O / Increased Multiprogramming\n");
    return 0;
}
```

Output

```
========== BEFORE OPTIMIZATION ==========
Total CPU Time      : 64 ms
Total I/O Wait Time : 78 ms
CPU Utilisation     : 45.07%

Reason:
Processes frequently move to Waiting state due to I/O.

========== AFTER OPTIMIZATION ==========
Reduced I/O Wait Time : 39 ms
CPU Utilisation       : 62.14%

Optimization Applied:
1. Faster I/O Devices
2. Asynchronous I/O / Increased Multiprogramming


=== Code Execution Successful ===
```

3. Memory Bottleneck Debugging

CODE :

```
#include <stdio.h>
int main()
{
    int codeSize = 5;
    int heapSize = 150;
    int stackPerThread = 1;
    int threads = 25;
    int ram = 2048;
    int totalMemory, optimizedMemory;
    int optimizedThreads = 10;
    int optimizedHeap = 100;
    printf("========== MEMORY BOTTLENECK DEBUGGING
==========\n\n");
    printf("Memory Map\n");
    printf("----------\n");
    printf("Code Size          : %d MB\n", codeSize);
    printf("Heap Size          : %d MB\n", heapSize);
    printf("Stack Per Thread   : %d MB\n", stackPerThread);
    printf("Number of Threads  : %d\n", threads);
    printf("Physical RAM       : %d MB\n", ram);
    totalMemory = codeSize + heapSize + (threads * stackPerThread);
    printf("\nTotal Memory Used : %d MB\n", totalMemory);
```

```
    if(totalMemory < ram)
    {
        printf("\nPhysical memory is sufficient.\n");
        printf("However, many threads access memory simultaneously.\n");
        printf("This causes frequent page faults and performance degradation.\n");
    }
    else
    {
        printf("\nMemory exceeds RAM.\n");
        printf("Heavy page faults will occur.\n");
    }
    printf("\n========== OPTIMIZATION ==========\n");
    optimizedMemory = codeSize + optimizedHeap +
                      (optimizedThreads * stackPerThread);
    printf("\nOptimization 1 : Reduce Heap Usage\n");
    printf("Heap Reduced To : %d MB\n", optimizedHeap);
    printf("\nOptimization 2 : Use Thread Pool\n");
    printf("Threads Reduced To : %d\n", optimizedThreads);
    printf("\nOptimized Memory Usage : %d MB\n", optimizedMemory);
    printf("\nPerformance Improved");
    printf("\nPage Faults Reduced");
    printf("\nMemory Access Faster");
    return 0;
}
```

OUTPUT :

========== MEMORY BOTTLENECK DEBUGGING ==========

Memory Map

Code Size : 5 MB

Heap Size : 150 MB

Stack Per Thread : 1 MB

Number of Threads : 25

Physical RAM : 2048 MB

Total Memory Used : 180 MB

Physical memory is sufficient.

However, many threads access memory simultaneously.

This causes frequent page faults and performance degradation.

========== OPTIMIZATION ==========

Optimization 1 : Reduce Heap Usage

Heap Reduced To : 100 MB

Optimization 2 : Use Thread Pool

Threads Reduced To : 10

Optimized Memory Usage : 115 MB

Performance Improved

Page Faults Reduced

Memory Access Faster

=== Code Execution Successful ===

EXECUTION :

main.c

```c
#include <stdio.h>
int main()
{
    int codeSize = 5;
    int heapSize = 150;
    int stackPerThread = 1;
    int threads = 25;
    int ram = 2048;
    int totalMemory, optimizedMemory;
    int optimizedThreads = 10;
    int optimizedHeap = 100;
    printf("========== MEMORY BOTTLENECK DEBUGGING ==========\n\n"
        );
    printf("Memory Map\n");
    printf("----------\n");
    printf("Code Size          : %d MB\n", codeSize);
    printf("Heap Size          : %d MB\n", heapSize);
    printf("Stack Per Thread   : %d MB\n", stackPerThread);
    printf("Number of Threads  : %d\n", threads);
```

Output

```
========== MEMORY BOTTLENECK DEBUGGING ==========

Memory Map
----------
Code Size          : 5 MB
Heap Size          : 150 MB
Stack Per Thread   : 1 MB
Number of Threads  : 25
Physical RAM       : 2048 MB

Total Memory Used : 180 MB

Physical memory is sufficient.
However, many threads access memory simultaneously.
This causes frequent page faults and performance degradation.

========== OPTIMIZATION ==========

Optimization 1 : Reduce Heap Usage
```

main.c

```c
    }
    else
    {
        printf("\nMemory exceeds RAM.\n");
        printf("Heavy page faults will occur.\n");
    }
    printf("\n========== OPTIMIZATION ==========\n");
    optimizedMemory = codeSize + optimizedHeap +
                      (optimizedThreads * stackPerThread);
    printf("\nOptimization 1 : Reduce Heap Usage\n");
    printf("Heap Reduced To : %d MB\n", optimizedHeap);
    printf("\nOptimization 2 : Use Thread Pool\n");
    printf("Threads Reduced To : %d\n", optimizedThreads);
    printf("\nOptimized Memory Usage : %d MB\n", optimizedMemory);
    printf("\nPerformance Improved");
    printf("\nPage Faults Reduced");
    printf("\nMemory Access Faster");
    return 0;
}
```

Output

```
Physical memory is sufficient.
However, many threads access memory simultaneously.
This causes frequent page faults and performance degradation.

========== OPTIMIZATION ==========

Optimization 1 : Reduce Heap Usage
Heap Reduced To : 100 MB

Optimization 2 : Use Thread Pool
Threads Reduced To : 10

Optimized Memory Usage : 115 MB

Performance Improved
Page Faults Reduced
Memory Access Faster

=== Code Execution Successful ===
```

4. Starvation Debugging

CODE :

```
#include <stdio.h>
#define MAX 10
struct Process
{
   int pid;
   int queue;
   int burst;
   int waiting;
   int completed;
};
int main()
{
   struct Process p[MAX];
   int n = 5;
   p[0] = (struct Process){1,1,3,0,0};
   p[1] = (struct Process){2,1,2,0,0};
   p[2] = (struct Process){3,2,4,0,0};
   p[3] = (struct Process){4,1,3,0,0};
   p[4] = (struct Process){5,3,5,0,0};
   int completed = 0;
   int cycle = 1;
```

```
printf("\n===== MULTILEVEL QUEUE SCHEDULER =====\n");
while(completed < n)
{
   printf("\nScheduling Cycle %d\n", cycle);
   int executed = 0;
   for(int i=0;i<n;i++)
   {
      if(!p[i].completed && p[i].queue==1)
      {
         printf("Executing P%d from Q1\n",p[i].pid);
         p[i].completed=1;
         completed++;
         executed=1;
         break;
      }
   }
   if(!executed)
   {
      for(int i=0;i<n;i++)
      {
         if(!p[i].completed && p[i].queue==2)
         {
            printf("Executing P%d from Q2\n",p[i].pid);
            p[i].completed=1;
```

```
            completed++;
            executed=1;
            break;
        }
    }
}
if(!executed)
{
    for(int i=0;i<n;i++)
    {
        if(!p[i].completed && p[i].queue==3)
        {
            printf("Executing P%d from Q3\n",p[i].pid);
            p[i].completed=1;
            completed++;
            executed=1;
            break;
        }
    }
}
for(int i=0;i<n;i++)
{
    if(!p[i].completed)
    {
```

```
            p[i].waiting++;
            if(p[i].queue==3 && p[i].waiting>=3)
            {
               printf("Aging Applied -> P%d moved Q3 -> Q2\n",p[i].pid);
               p[i].queue=2;
            }
            if(p[i].queue==2 && p[i].waiting>=5)
            {
               printf("Aging Applied -> P%d moved Q2 -> Q1\n",p[i].pid);
               p[i].queue=1;
            }
         }
      }
      cycle++;
   }
   printf("\nAll Processes Executed Successfully.\n");
   printf("Starvation Eliminated Using Aging.\n");
   return 0;
}
```

OUTPUT :

```
===== MULTILEVEL QUEUE SCHEDULER =====
Scheduling Cycle 1
Executing P1 from Q1
Scheduling Cycle 2
Executing P2 from Q1
Scheduling Cycle 3
Executing P4 from Q1
Aging Applied -> P5 moved Q3 -> Q2
Scheduling Cycle 4
Executing P3 from Q2
Scheduling Cycle 5
Executing P5 from Q2
All Processes Executed Successfully.
Starvation Eliminated Using Aging.
=== Code Execution Successful ===
```

EXECUTION :

main.c — Share — Run — Output — Clear

```c
#include <stdio.h>
#define MAX 10
struct Process
{
    int pid;
    int queue;
    int burst;
    int waiting;
    int completed;
};
int main()
{
    struct Process p[MAX];
    int n = 5;
    p[0] = (struct Process){1,1,3,0,0};
    p[1] = (struct Process){2,1,2,0,0};
    p[2] = (struct Process){3,2,4,0,0};
    p[3] = (struct Process){4,1,3,0,0};
    p[4] = (struct Process){5,3,5,0,0};
```

```
===== MULTILEVEL QUEUE SCHEDULER =====

Scheduling Cycle 1
Executing P1 from Q1

Scheduling Cycle 2
Executing P2 from Q1

Scheduling Cycle 3
Executing P4 from Q1
Aging Applied -> P5 moved Q3 -> Q2

Scheduling Cycle 4
Executing P3 from Q2

Scheduling Cycle 5
Executing P5 from Q2
```

Programiz C Online Compiler — Programiz PRO ❯

main.c — Share — Run — Output — Clear

```c
            {
                printf("Aging Applied -> P%d moved Q3 -> Q2\n"
                    ,p[i].pid);
                p[i].queue=2;
            }
            if(p[i].queue==2 && p[i].waiting>=5)
            {
                printf("Aging Applied -> P%d moved Q2 -> Q1\n"
                    ,p[i].pid);
                p[i].queue=1;
            }
        }
    }
    cycle++;
}
printf("\nAll Processes Executed Successfully.\n");
printf("Starvation Eliminated Using Aging.\n");
return 0;
}
```

```
Scheduling Cycle 2
Executing P2 from Q1

Scheduling Cycle 3
Executing P4 from Q1
Aging Applied -> P5 moved Q3 -> Q2

Scheduling Cycle 4
Executing P3 from Q2

Scheduling Cycle 5
Executing P5 from Q2

All Processes Executed Successfully.
Starvation Eliminated Using Aging.


=== Code Execution Successful ===
```

5. Context-Switch Overhead Debugging

CODE :

```
#include <stdio.h>
int main()
{
   int n, i;
   printf("Enter number of processes: ");
   scanf("%d",&n);
   int bt[20], temp[20];
   printf("\nEnter Burst Time of each process:\n");
   for(i=0;i<n;i++)
   {
      printf("P%d : ",i+1);
      scanf("%d",&bt[i]);
   }
   int quantum = 1;
   int completed = 0;
   int contextSwitch = 0;
   for(i=0;i<n;i++)
      temp[i]=bt[i];
   while(completed<n)
   {
```

```
    for(i=0;i<n;i++)
    {
        if(temp[i]>0)
        {
            if(temp[i]>quantum)
            {
                temp[i]-=quantum;
                contextSwitch++;
            }
            else
            {
                temp[i]=0;
                completed++;
                contextSwitch++;
            }
        }
    }
}
printf("\n========== BEFORE OPTIMIZATION ==========\n");
printf("Time Quantum : %d ms\n",quantum);
printf("Context Switches : %d\n",contextSwitch);
quantum=4;
completed=0;
contextSwitch=0;
```

```
for(i=0;i<n;i++)
   temp[i]=bt[i];
while(completed<n)
{
   for(i=0;i<n;i++)
   {
      if(temp[i]>0)
      {
         if(temp[i]>quantum)
         {
            temp[i]-=quantum;
            contextSwitch++;
         }
         else
         {
            temp[i]=0;
            completed++;
            contextSwitch++;
         }
      }
   }
}
printf("\n========== AFTER OPTIMIZATION ==========\n");
printf("Time Quantum : %d ms\n",quantum);
```

```
    printf("Context Switches : %d\n",contextSwitch);
    if(contextSwitch<20)
        printf("\nOptimization Successful.");
    else
        printf("\nContext Switching Still High.");
    return 0;
}
```

OUTPUT :

```
Enter number of processes: 6
Enter Burst Time of each process:
P1 : 4
P2 : 6
P3 : 5
P4 : 2
P5 : 9
P6 : 7
========== BEFORE OPTIMIZATION ==========
Time Quantum : 1 ms
Context Switches : 33
========== AFTER OPTIMIZATION ==========
Time Quantum : 4 ms
Context Switches : 11
Optimization Successful.
```

=== Code Execution Successful ===

EXECUTION :

main.c

```c
#include <stdio.h>
int main()
{
    int n, i;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    int bt[20], temp[20];
    printf("\nEnter Burst Time of each process:\n");
    for(i=0;i<n;i++)
    {
        printf("P%d : ",i+1);
        scanf("%d",&bt[i]);
    }
    int quantum = 1;
    int completed = 0;
    int contextSwitch = 0;
    for(i=0;i<n;i++)
        temp[i]=bt[i];
    while(completed<n)
```

Output

```
Enter number of processes: 6

Enter Burst Time of each process:
P1 : 4
P2 : 6
P3 : 5
P4 : 2
P5 : 9
P6 : 7

========== BEFORE OPTIMIZATION ==========
Time Quantum : 1 ms
Context Switches : 33

========== AFTER OPTIMIZATION ==========
Time Quantum : 4 ms
Context Switches : 11

Optimization Successful.
```